

Projektdokumentation „Retro Terminal“

Ein Atmega328 Aufbau zur Verarbeitung von PS/2 Inputs und Ausgabe über VGA

Ein Projekt von Jule Jäger, Natalie Weigand und Benjamin Blaga

15.05.2026



GitHub-Repo zum „Retro Terminal“

Ziel des Projekts:

Das Ziel des Projektes ist der Aufbau und die Inbetriebnahme einer eigens entworfenen Schaltung zum Einlesen, Auswerten und Ausgeben von PS/2 Inputs über VGA. Umgesetzt werden soll dies mithilfe von zwei ATmega328-P Mikrocontroller, vorgeladen mit dem Arduino Uno Bootloader. Dabei soll die Modularität für zukünftige Erweiterungen ermöglicht werden sowie ergänzende Funktionen wie z.B. ein Buzzer bereitgestellt werden.

Hauptfunktionen:

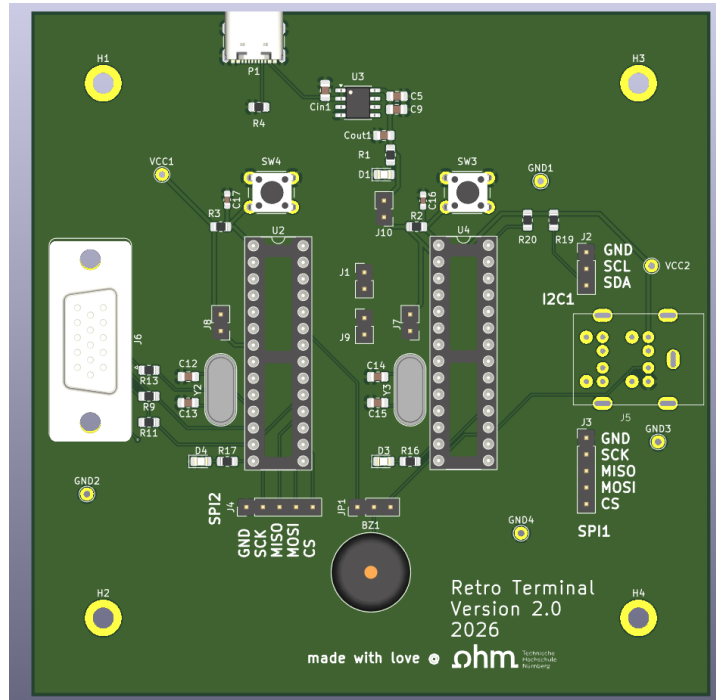
- Einlesen von Benutzerinput mithilfe einer Tastatur über den PS/2-Standard
- Ausgabe von eingelesenen Benutzerinput über monochromes VGA
- Ansteuerung eines Buzzers für akustische Funktionen
- Schaltungsaufbau ermöglicht einfaches Debugging bei Änderungsbedarf

Projektkomponenten:

- Platine
- Software

In dieser Dokumentation wird der Aufbau, die Systemkomponenten und die Inbetriebnahme erklärt.

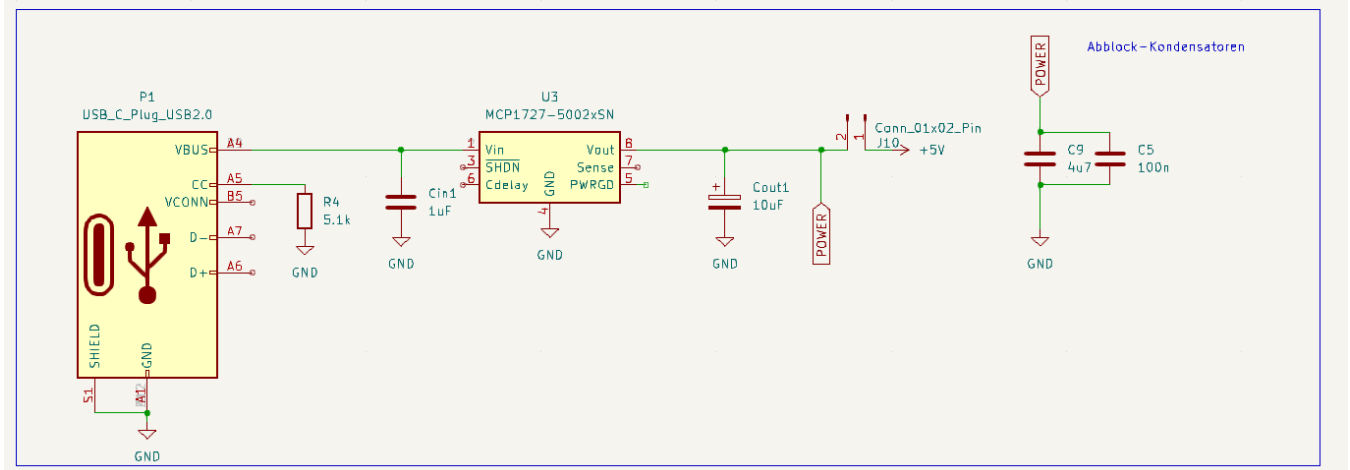
Platine



3D-Render der Platine, erstellt in KiCad (15.05.2026)

Stromversorgung

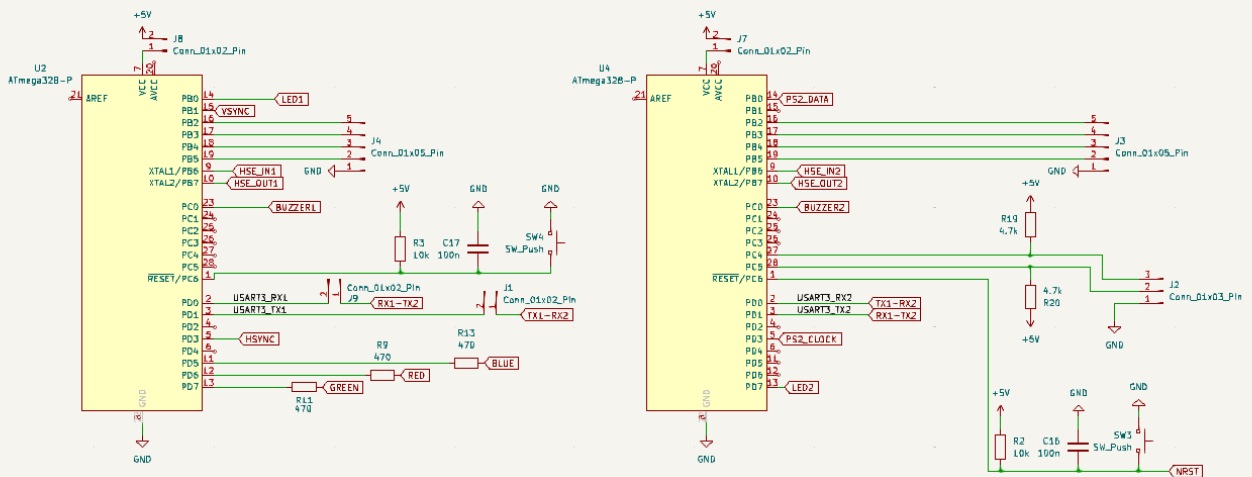
Power



Die Platine wird versorgt mit einem USB-C Kabel über ein USB-C konformen Verbindungsstecker. Der Widerstand R4 vermittelt dem USB-C Host (Versorger) dass diese Platine ein Sink ist und Standard-USB-Spannung erwartet wird, in diesem Fall 5V. Weiterhin wird, um Überspannung zu verhindern ein LDO-Spannungsregler vom Typ MCP1727-5002 benutzt, um eine konstante Spannung von 5V zu garantieren sowie ein Maximalstrom von 1.5A. Zum Schutz der Mikrocontroller wurden Abblock-Kondensatoren platziert sowie Jumper-Pins, falls eine alternative Spannungsversorgung benötigt wird.

Mikrocontroller

MCU



Die Platine nutzt 2x Microchip ATmega328-P Mikrocontroller in der DIP-28 Bauform. Diese werden betrieben in einer Minimalausführung: Es werden lediglich Spannungsversorgung sowie Takt mithilfe von Standardquarzen (siehe unten) zugeführt. Um das Debugging zu erleichtern, haben die Mikrocontroller jeweils einzelne Reset-Knöpfe sowie Jumper-Pins zum optionalen Ausschalten der einzelnen Mikrocontroller.

Herausgeführt wurden die benötigten GPIO-Pins für die benötigten Funktionalitäten, namentlich:

- Verbindungen für VGA-Funktionalität (Pins gekennzeichnet mit „RED“, „GREEN“, „BLUE“, „VSYNC“, „HSYNC“)
- Verbindungen für PS/2-Funktionalität (Pins gekennzeichnet mit „PS2_CLOCK“, „PS2_DATA“)
- Verbindungen für die serielle Kommunikation zwischen den beiden ATmega328-P Mikrocontrollern (Pins gekennzeichnet mit „TX1-RX2“, „RX1-TX2“, „RX1-TX2“, „TX2-RX2“)
- Verbindungen für Buzzer-Funktionalität (Pins gekennzeichnet mit „BUZZER1“, „BUZZER2“)
- Verbindungen für serielle Schnittstellen für zukünftige Erweiterungen (Pins verbunden mit „J4“, „J3“ und „J2“)
- Verbindungen für Status-LEDs (Pins gekennzeichnet mit „LED1“, „LED2“)

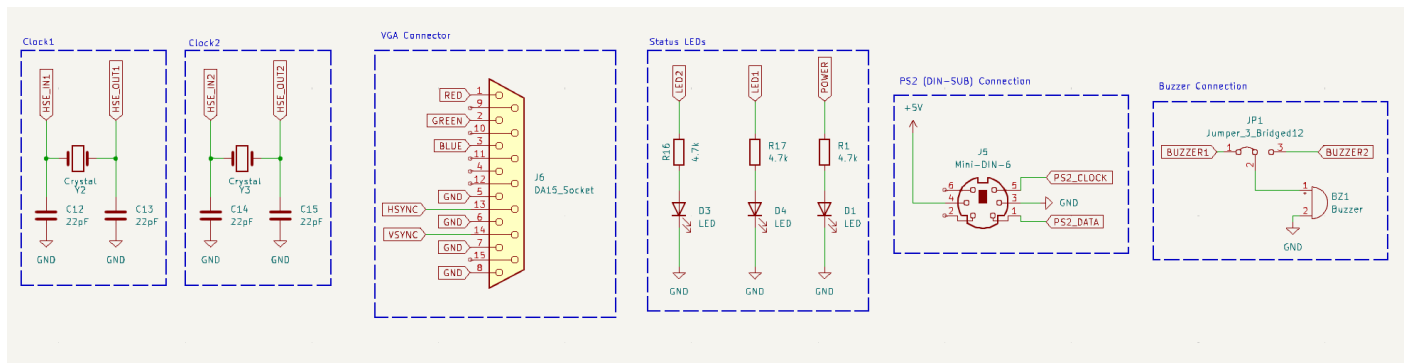
(Verbindungen zur allgemeinen Stromversorgung werden hier nicht berücksichtigt)

Es sind Jumper-Pins vorhanden um die bidirektionale Kommunikation über UART zwischen den Mikrocontrollern einzuschränken/ modifizieren zu können.

Beide Mikrocontroller sind in der Lage den Buzzer anzusteuern, falls benötigt.

„J2“ ist als I2C-Steckverbindung, „J3“ und „J4“ als SPI-Steckverbindungen vorgesehen.

Diverse Systemkomponenten

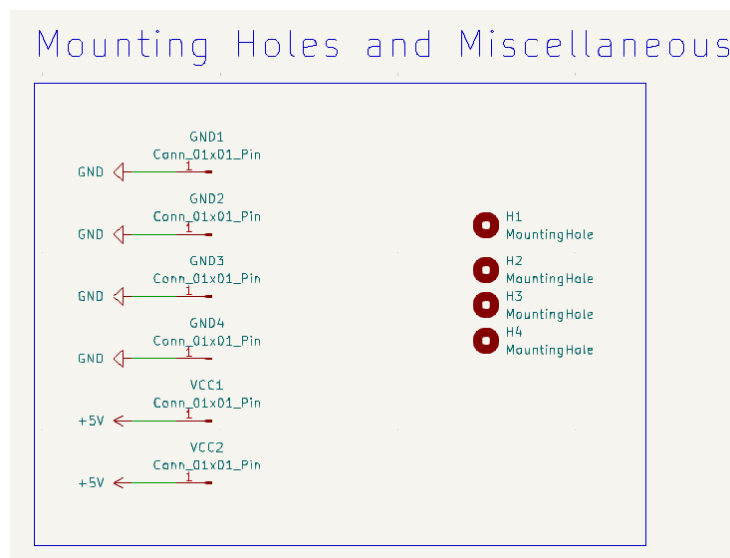


Zu den diversen Systemkomponenten zählen die Taktgeber in Form von Standardquarzen (sowie deren benötigte Keramikkondensatoren), die VGA-Steckverbindung, die Status-LEDs, die DIN6-SUB Steckverbindung für eine PS/2 Tastatur sowie der Buzzer.

Die Mikrocontroller haben jeweils eine Status-LED zum freien Ansteuern, weiterhin gibt es eine Status-LED zur Überwachung der Stromversorgung.

Der Buzzer kann über Jumper-Pins einem Mikrocontroller zur Ansteuerung zugewiesen werden.

Mechanische Bauteile



Die „Mounting Holes“ wurden platziert, um eine einfachere Montage zu ermöglichen. Weiterhin wurden, wie in der Praxis üblich, Vias bzw. Pins herausgeführt, um Betriebsspannungen messen zu können.

Software

```

/*****
 *      Arduino TinyBasic VGA PC
 *
 *      Version 1 - February the 4th, 2018, Written by Roberto Melzi
 *
 *      Arduino TinyBasic VGA PC is a simple, 8-bit retro-PC, composed
 *      by two Arduino board. One runs TinyBasic and the PS2 library,
 *      while the other runs a code, based on the VGAX library, to
 *      generate the output signal for a VGA monitor.
 *
 *      Both TinyBasic and VGAX requires Arduino IDE version 1.6.4
 *      VGAX library does not work for newer or older software
 *      versions.
 *
 *      VGAX Library written By Sandro Maffiodo aka Smaffer.
 *      https://github.com/smaffer/vgax
 *      See also the Arduino forum:
 *      http://forum.arduino.cc/index.php?topic=328631.0
 *      and on Instructables:
 *      http://www.instructables.com/member/Rob%20Cai/instructables/
 *****/

```

Projektbeschreibung „TinyBasic VGA PC“ von Roberto Melzi

Dieses Projekt benutzt eine abgeänderte Version des „[Arduino TinyBasic VGA PC](#)“- Codes von Roberto Melzi. Dieses basiert wiederum auf [der VGAX Library](#) von Sandro Maffiodo, welches die Basis vieler Arduino-basierter VGA-Projekte darstellt. Diese Library stellt VGA-Funktionalität auf Hardware-Timer-Basis bereit und ermöglicht zusätzlich die Darstellung von Bildern. Aufgrund vom technischen Umfang einer Implementierung der VGA-Kommunikation benutzt dieses Projekt die VGAX-Schnittstelle bzw. eine modifizierte Version des „TinyBasic VGA PC“ Projektes zur Bereitstellung der erfordernten Funktionalität.

Die benötigte Rechenlast zur Ausgabe von VGA-Videosignalen ist nicht besonders hoch, jedoch ist z.B. bei der Darstellung von Bildern ein Framebuffer notwendig. Dies füllt den beschränkt vorhandenen RAM des ATmega328P fast vollständig und schränkt die Größe des weiteren Codes ein. Deshalb wurde die PS/2 Kommunikation ausgelagert auf einen zweiten ATmega328P, um diesen zu entlasten. Somit wird der PS/2 Benutzerinput von einem ATmega328P eingelesen, verarbeitet und per „Serial.print()“ (UART) an den anderen ATmega328P gesendet.

```

if (Serial.available() > 0) {
  myNumber = Serial.read();
  vgaPrintAscii(byte(95), x, y, 0);
}

```

Screenshot vom Code des VGA-ausgebenden Atmega328P

Dieser empfängt die gelesenen bytes über Serial.read() und gibt diese daraufhin aus.

PS/2 Handling & BASIC-Interpreter

Der am PS/2 Port angeschlossene ATmega328P verarbeitet die eingelesenen Tastenschläge der PS/2-Tastatur sowie interpretiert Befehle, um Programmspeicher auf dem VGA-treibenden ATmega328P nicht zu vereinnahmen. Hierfür benutzt das geladene Programm die „PS2Keyboard“-Library die in der Arduino IDE über „Manage Libraries“ einzeln geladen werden kann. Grundsätzlich sind alle benötigten PS/2-Funktionen anhand dieser Library implementiert und benötigen lediglich die Konfiguration des Data-Pins sowie des IRQ-Pins.

Das geladene Programm beinhaltet eine Art BASIC-Interpreter:

```
// Keyword table and constants - the last character has 0x80 added to it
const static unsigned char keywords[] PROGMEM = {
  'L','I','S','T'+0x80,
  'L','O','A','D'+0x80,
  'N','E','W'+0x80,
  'R','U','N'+0x80,
  'S','A','V','E'+0x80,
  'N','E','X','T'+0x80,
  'L','E','T'+0x80,
  'I','F'+0x80,
  'G','O','T','O'+0x80,
  'G','O','S','U','B'+0x80,
  'R','E','T','U','R','N'+0x80,
  'R','E','M'+0x80,
  'F','O','R'+0x80,
  'I','N','P','U','T'+0x80,
  'P','R','I','N','T'+0x80,
  'P','O','K','E'+0x80,
  'S','T','O','P'+0x80,
  'B','Y','E'+0x80,
  'F','I','L','E','S'+0x80,
  'M','E','M'+0x80,
  '?'+ 0x80,
  '\'+ 0x80,
  'A','W','R','I','T','E'+0x80,
  'D','W','R','I','T','E'+0x80,
  'D','E','L','A','Y'+0x80,
  'E','N','D'+0x80,
  'R','S','E','E','D'+0x80,
  'C','H','A','I','N'+0x80,
```

Screenshot des Codes mit den erwähnten Keywords

Diese können benutzt werden, um BASIC-artigen Code zu schreiben und lokal auszuführen. Weiterhin sind bestimmte Speicherbereiche für Variablen nach BASIC-Standard vorzufinden (gekennzeichnet als Variablen „A“ bis „Z“, intern gespeichert als *short int*).

Die unterstützten Rechenoperationen des Interpreters sind Addition („+“), Subtraktion („-“), Multiplikation („*“) und Division („/“). Weiterhin sind Vergleichsoperatoren vorzufinden: gleich („=“), ungleich („<>“ oder „!=“), kleiner („<“), größer („>“), kleiner gleich („<=“) und größer gleich („>=“).

Ebenfalls sind bereits primitive Funktionen implementiert.

```
const static unsigned char func_tab[] PROGMEM = {  
    'P','E','E','K'+0x80,  
    'A','B','S'+0x80,  
    'A','R','E','A','D'+0x80,  
    'D','R','E','A','D'+0x80,  
    'R','N','D'+0x80,  
    0  
};
```

Zu erwähnen ist hierbei, dass z.B. die Funktion „PEEK()“, welche im BASIC-Standard zum Lesen von Speicherzellen gedacht ist, hierbei nur auf den beim Setup allokierten Speicher zugreift. Somit erfolgt lediglich ein Zugriff auf Programmspeicher und nicht ein Zugriff auf ein CPU-Register des ATmega328P. Die Funktionen „AREAD()“ und „DREAD()“ implementieren die Arduino-Funktionen „analogRead()“ und „digitalRead()“, welche jeweils das Einlesen von analogen und digitalen Werten möglich machen. Analog dazu sind ebenfalls die Funktionen „AWRITE()“ und „DWRITE()“ implementiert.

Ablauf des Interpreters

setup()

Initialisiert:

- Serielle Schnittstelle
- (PS/2) Tastatur
- EEPROM (optional)
- SD-Karte (optional)

loop()

Hauptinterpreter:

- Speicher initialisieren
- BASIC-Eingabe lesen
- Zeilen speichern
- Befehle interpretieren
- Programme ausführen

Beispiel:

```
10 PRINT 123  
20 GOTO 10  
30 RUN|
```

Screenshot eines Beispielprogramms

Der Interpreter reagiert auf Zahlen am Anfang einer Zeile als ein Marker für BASIC-Code. Somit wird der BASIC-Code direkt gespeichert und ohne Zahl am Zeilenanfang direkt ausgeführt

```
123
123
123
123
123
123
123
123
123
123
123
123
123
123
123
|
```

Erwartete Ausgabe des Codes

Funktionsablauf:

Der Interpreter erkennt „PRINT“ über **scantable(keywords)** und springt zu einem Funktionshandler. Dieser ruft die Funktion **print_quoted_string()** auf und der eingegebene String (welcher eingespeichert wurde) wird über **outchar()** ausgegeben, welches lediglich ein Wrapper für **Serial.write()** ist. Somit wird eingelesener Code/Input zuletzt über **Serial.write()** und somit über UART an den VGA-ATmega328P gesendet.

Durch Drücken von „Shift“ + „7“ kann die Farbe des CLI geändert werden (grün, rot, gelb). Durch „Ctrl“ + „C“ kann eine Endlosschleife unterbrochen werden.

Bekannte Probleme:

Aufgrund des begrenzten Speicherplatzes des ATmega328P kommt es schnell zu einem Überlauf. So können ohne Resets Befehle oft nicht verarbeitet werden (Benachrichtigung mit „What?“ oder „How?“). Ebenfalls werden die Tastenschläge nach US-Layout eingelesen, was bei nicht standardisierten PS/2 Tastaturen zu Problemen führen kann. Der Testaufbau erfolgte mit einer IBM-Model M Tastatur.

Benutzung der Platine

Die Platine verfügt über einen USB-C Anschluss zur Stromversorgung. Programmiert werden die ATmega328P über ein Arduino UNO Board, da kein ISP vorhanden ist. Dazu werden die ATmega328P aus den jeweiligen Sockeln entfernt, in die Sockel auf dem Arduino UNO Board eingesteckt und wie gewohnt programmiert. Dazu sind im GitHub Repository zu diesem Projekt die „.ino“ Programme hinterlegt: Einmal „ps2_atmega328p“ sowie „vga_atmega328p“. Diese werden auf die dazugehörigen ATmega328P geladen. Danach ist das System startbereit. Die „POWER“ Status-LED kann beobachtet werden, um eine funktionierende Stromversorgung zu überprüfen. Zum Debuggen können die Jumper zur Stromversorgung der ATmega328P („J8“ und „J7“) einzeln entfernt werden, genauso wie die Verbindung zum Buzzer „BUZZER1“ („JP1“). Ebenfalls kann die UART-Kommunikation von bidirektional auf unidirektional (Master und Slave) durch Jumper („J1“, „J9“) umfunktioniert werden.

Wichtig hierbei ist anzumerken, dass die verwendeten ATmega328P den Arduino UNO Bootloader geladen haben müssen, damit dieses System funktioniert. Jedoch sind die meisten ATmega328P bereits damit vorprogrammiert ab Werk.

Die Farbe „BLUE“ für den VGA-Standard ist nicht angeschlossen, aufgrund der Auslegung auf einen monochromen Betrieb in der Library „VGAX“ auf der das TinyBasic Projekt basiert. Jedoch kann, falls gewünscht, der Pin überbrückt werden und die Library umprogrammiert werden, da ein geeigneter Pin an PORTD hierfür ausgelegt wurde.

Dieses System kann erweitert werden mit einem EEPROM und/oder SPI und I2C Modulen. Jedoch muss zur Erweiterung mit serieller Kommunikation das Timing berücksichtigt werden. Hierzu bitte die Hinweise im „VGAX“ [GitHub Repo](#) berücksichtigen, falls die Erweiterungen auf dem „VGA“ Atmega328P erfolgen.

Implementierung auf STM32-Basis

Grundsätzlich lässt sich dieses Projekt mit derselben Funktionalität auch auf einem STM32 implementieren. VGA-Libraries auf STM32-Basis sind vorhanden, jedoch sind die meisten bereits veraltet und nicht für die im MCT-Labor zu findenden STM32-Geräte vorgesehen. Somit wäre ein Port auf beispielsweise die STM32L4-Reihe notwendig.

Zu beachten ist, dass die „Totzeiten“, also Front- und Backporch, essenziell sind, um Fragmentierungen im Signal zu verhindern. Dafür müssen Timer aneinandergekettet werden; Softwaretimer kommen aufgrund der erforderlichen Synchronität nicht infrage. Eine Implementierung ist jedoch auch **ohne DMA** möglich.

Der BASIC-Interpreter kann von der Logik her übernommen werden, müsste jedoch auf STM32-Cube IDE konformen Code umgeschrieben werden.

Folgend sind Ressourcen gelistet, die den Prozess näher beschreiben:

https://mikrocontroller.bplaced.net/wordpress/stm32f4/komplette-library-liste-stm32f4/35-vga_screen-library-stm32f4/

<https://github.com/RoCorbera/BlueVGA>

<https://martin.hinner.info/vga/timing.html>

<https://github.com/MR-Addict/STM32-HAL-PS2-Library>

Quellen:

<https://www.instructables.com/Arduino-Basic-PC-With-VGA-Output/> (abgerufen am 20.05.2026)

<https://github.com/smaffer/vgax> (abgerufen am 20.05.2026)

<https://www.circuito.io/blog/arduino-uno-pinout/> (abgerufen am 20.05.2026)

<https://github.com/benjemiah/projektarbeit> (abgerufen am 20.05.2026)